- Include keyword extraction and topic modeling as supplementary features.
- Allow users to highlight specific sections for prioritization during summarization.
- Offer downloadable summary reports in formats like PDF or Word.

7. Scalability and Optimization:
    - Use GPU acceleration and model quantization techniques to optimize performance.
    - Employ distributed computing for handling large-scale summarization tasks.
    - Implement caching mechanisms for frequently requested documents or articles.
    - Ensure compliance with data privacy laws like GDPR when handling sensitive content.

Outcome:

By implementing this project, users can quickly grasp the essence of lengthy texts without spending hours reading. The tool has applications in journalism, education, research, and business, making it a valuable asset for summarizing reports, articles, or meeting notes. Its advanced features and real-time deployment make it suitable for both individual and enterprise-level use cases.

Example Code:

```
import nltk
from nltk.tokenize import sent_tokenize from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity import numpy as np

# Step 1: Preprocess Text
def preprocess_text(text):
    sentences = sent_tokenize(text) return sentences

# Step 2: Calculate Sentence Scores
def calculate_tfidf(sentences):
```